

Keeping Your Changes Across Weekly Code Exports

A vendor-branch workflow in Git, so your edits get replayed onto each new export automatically instead of being retyped by hand.

WHAT THIS DOCUMENT COVERS

1. The problem, precisely
2. The mental model: two branches
3. One-time setup
4. The weekly routine
5. When Git hits a conflict
6. Habits that keep the weekly rebase cheap
7. Command cheat sheet

01 The problem, precisely

You receive a snapshot of your friend's repository once a week. It arrives with no history — just files. You apply your own modifications to make it work for you. Next week another snapshot arrives, and because it's a fresh set of files, your modifications are simply gone.

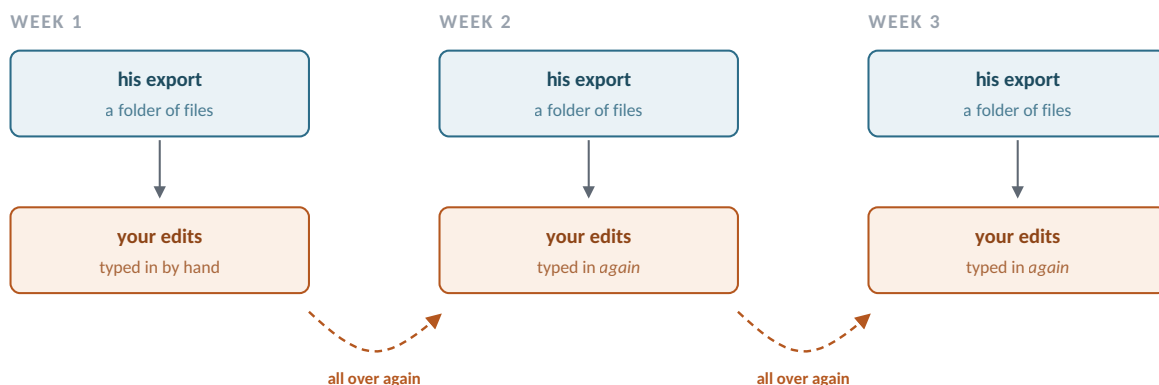


Figure 1. Redoing the same work every week. The effort scales with the number of exports, not with the number of changes you actually have.

The fix is to stop treating each export as a fresh start. Instead, put the exports in a Git repository on a branch of their own, keep your changes as commits on a second branch, and let Git re-apply your commits on top of each new export. Git is very good at exactly this; the operation is called a **rebase**, and the pattern is known as a **vendor branch**.

02 The mental model: two branches

You will keep exactly two branches, and they never mix roles:

- **vendor** — holds nothing but his exports. One commit per export. You never edit files on this branch.
- **additions** — holds your changes, as commits sitting on top of whatever the latest export is.

Each week you add a commit to **vendor** and then rebase **additions** onto it. Rebasing means Git takes your commits off, fast-forwards to the new export, and replays your commits one at a time on the new base.

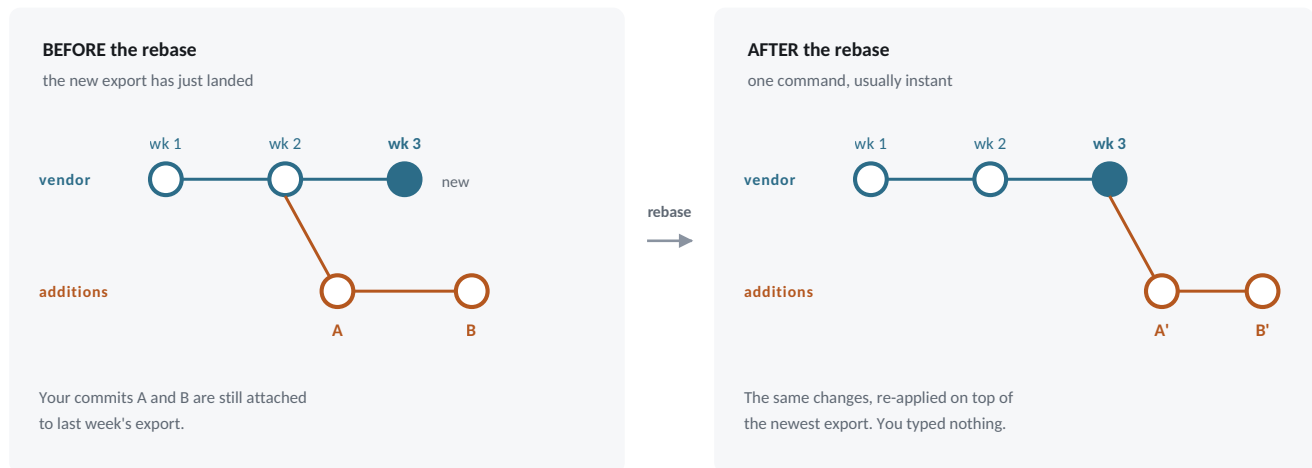


Figure 2. A rebase detaches your commits, moves the base forward to the newest export, and replays them. **A'** is the same change as **A**, adjusted to fit the new code.

Blue is his code, which you never edit. **Orange** is yours, which never gets mixed into his commits.

WHY REBASE RATHER THAN MERGE?

Merging would also combine the two branches, but it adds a merge commit every week and your changes end up buried in an ever-deeper history. Rebasing keeps your customizations as a short, readable stack sitting on top of the current export — which is also what makes `git diff vendor..additions` a reliable answer to "what exactly have I changed?"

03 One-time setup

Do this once, with whichever export you have on hand. The commands assume you're in a terminal and have Git installed.

Step 1 — Create the repository from the current export

```
mkdir project && cd project
git init

# extract this week's export so its files sit directly in this folder
unzip ~/Downloads/export.zip
# if the zip made a nested folder, move its contents up one level first

git add -A
git commit -m "vendor: export 2026-09-17"
```

Step 2 — Name that branch `vendor` and branch off it

```
git branch -m vendor      # this branch only ever holds his code
git switch -c additions   # your changes live here
```

Step 3 — Make your changes and commit them

```
# edit files as you normally would, then:
git add -A
git commit -m "point config at my database"

# commit each distinct change separately – this matters, see section 6
git commit -am "disable the telemetry hook"
```

Step 4 — Turn on conflict memory

```
git config rerere.enabled true
```

`rerere` stands for "reuse recorded resolution." With it on, Git remembers how you resolved a particular conflict and silently applies the same resolution the next time that identical conflict appears. In this workflow the same conflicts recur constantly, so this single setting saves a great deal of time.

04 The weekly routine

This is the part you repeat. Five steps, typically under a minute.

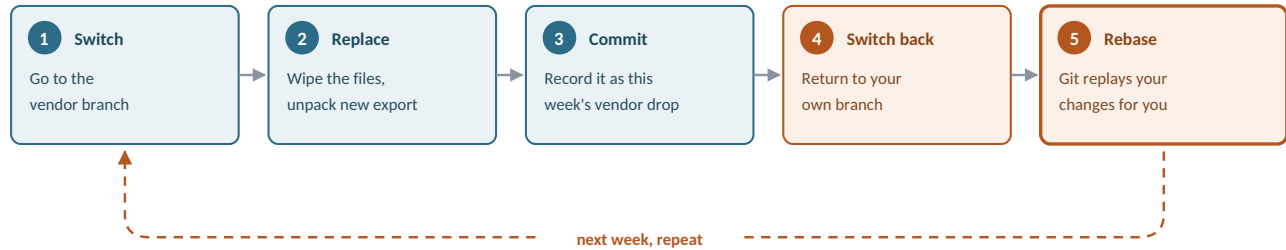


Figure 3. The weekly cycle. Steps 1–3 only ever touch his code; steps 4–5 only ever touch yours.

The commands

```
# 1. move to the vendor branch
git switch vendor

# 2. empty the working tree, but keep the .git folder
find . -mindepth 1 -maxdepth 1 -not -name .git -exec rm -rf {} +

# ...then unpack this week's export into place
unzip -q ~/Downloads/export.zip
# for a tarball with a top-level folder, use:
# tar xf ~/Downloads/export.tar.gz --strip-components=1 -C .

# 3. record it
git add -A
git commit -m "vendor: export $(date +%F)"

# 4-5. go back to your branch and replay your work onto the new base
git switch additions
git rebase vendor
```

WHY WIPE THE FOLDER IN STEP 2

If you only unzip over the top, files your friend *deleted* this week would stay behind forever, and your repository would drift away from what he actually has. Emptying the tree first means `git add -A` records deletions correctly. The `-not -name .git` is what keeps your history safe — check that part of the command carefully before running it.

You can save the whole weekly sequence as a shell script, say `sync.sh`, taking the export path as an argument. After a few weeks, the routine collapses into `./sync.sh ~/Downloads/export.zip`.

05 When Git hits a conflict

A conflict only happens when your friend edited the same lines you edited. If he worked elsewhere in the file — even elsewhere in the same function — Git merges silently and you'll see nothing at all.

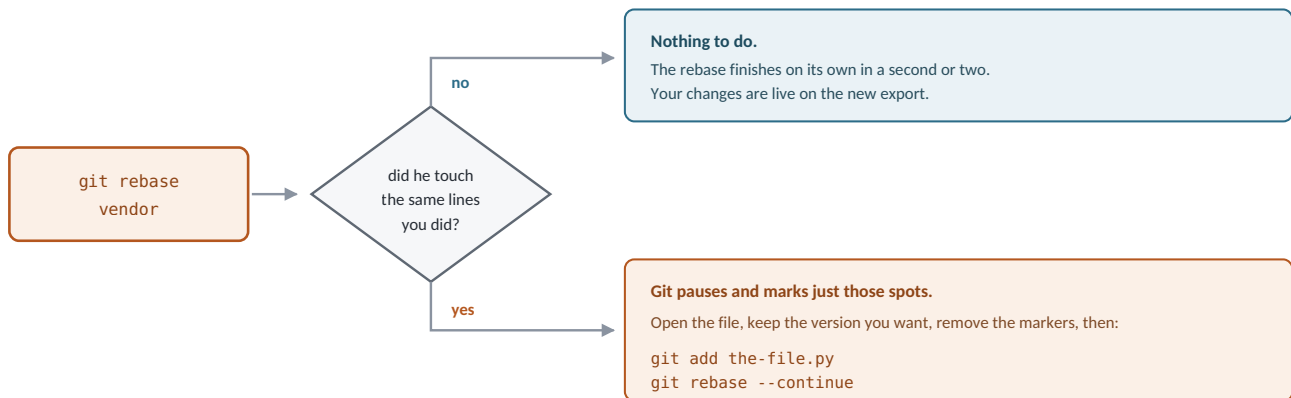


Figure 4. The two outcomes of a weekly rebase. A conflict is scoped to the overlapping lines, not the whole file or the whole codebase.

What a conflict looks like in the file

```
<<<<<< HEAD          <- his new version
timeout = 30
=====
timeout = 300          <- your version
>>>>>> disable the telemetry hook
```

Delete the three marker lines and whatever you don't want, leaving the code you intend to keep. Then `git add` that file and run `git rebase --continue`. Repeat if another commit conflicts.

Useful escape hatches

<code>git rebase --abort</code>	Cancel the whole rebase and go back to exactly where you were. Nothing is lost. Useful when you'd rather deal with it tomorrow.
<code>git rebase --skip</code>	Drop the commit being replayed. Correct when your friend has since implemented the change himself, making yours redundant.
<code>git diff</code>	During a pause, shows only the conflicted regions still needing attention.
<code>git status</code>	Tells you which commit is being applied and which files are blocking it.

06 Habits that keep the weekly rebase cheap

The workflow above always works, but these four habits are what make it take seconds instead of an afternoon.

Keep each change in its own small commit

One commit per logical change, with a message that says what it does. When a commit stops applying cleanly, Git names it — so you immediately know which of your customizations is affected, rather than facing one giant blob of unrelated edits.

Prefer adding files over editing them

A brand-new file can never conflict, because his export doesn't contain it. Whenever you have the choice:

- **Configuration:** put your values in a separate file the application reads (a local override, an `.env`, a `config.local.*`) rather than editing his config in place.
- **Behavior:** a wrapper, a subclass, or a small adapter module in a new file beats rewriting the body of his function.
- **Build and tooling:** your own script that calls his, rather than modifications inside his.

Every change you can express this way is one that will never conflict again.

Turn mechanical, repetitive edits into a script

If a change is purely mechanical — swapping a hostname everywhere, renaming a constant across many files — a script is more durable than a commit. Scripts don't conflict; they just run again.

```
# apply-additions.sh
set -e
grep -rl 'api.his-server.com' . --include='*.py' \
  | xargs sed -i 's/api.his-server.com/api.internal.example/g'
```

Commit the script itself on `additions`, and run it after each rebase.

Let Git remember your conflict resolutions

You enabled `rerere` in setup. It pays off from about the third week onward: recurring conflicts start resolving themselves, and you only look at genuinely new ones. You can confirm it's working when a rebase reports that it reused a recorded resolution.

ADD A .GITIGNORE EARLY

Build output, dependency folders, virtual environments, and editor files should be listed in `.gitignore` on the `additions` branch. Otherwise step 2 of the weekly routine produces enormous, noisy commits and the rebase has more to chew on than it needs.

07 Command cheat sheet

WHAT YOU WANT	COMMAND
See which branch you're on	<code>git status -sb</code>
Go to the vendor branch	<code>git switch vendor</code>
Go to your branch	<code>git switch additions</code>
Replay your work on the newest export	<code>git rebase vendor</code>
Continue after fixing a conflict	<code>git add FILE</code> then <code>git rebase --continue</code>
Back out of a rebase entirely	<code>git rebase --abort</code>
List only your own changes	<code>git log --oneline vendor..additions</code>
See exactly what you changed vs. his code	<code>git diff vendor..additions</code>
See what he changed this week	<code>git log -p -1 vendor</code>
Reorder, squash, or drop your commits	<code>git rebase -i vendor</code>
Recover a branch you moved by mistake	<code>git reflog</code> , then <code>git reset --hard HEAD@{N}</code>

Two of these are worth learning early. `git diff vendor..additions` is the complete, always-current answer to "what have I actually changed?" — the documentation of your customizations that you'd otherwise have to maintain by hand. And `git reflog` means almost no mistake in this workflow is permanent.

The idea to hold onto is that your changes should live as *commits*, not as edits you remember making — commits can be replayed; memory can't.